

Skillab

AI Agent Development

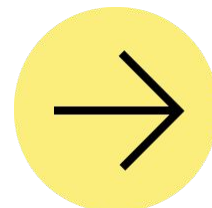
Richard Șerban Francu

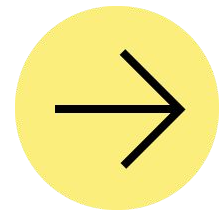
Senior AI Engineer @UCE Systems



Programa cursului

1. Noțiuni de bază — API-uri LLM și primul agent
2. Instrumente, prompturi și observabilitate
3. Procesarea și extragerea documentelor
4. Stocarea bazelor de date și RAG
5. LangGraph — Stare și fluxuri de lucru
6. Orchestrarea multi-agent
7. Optimizarea ML — Dincolo de apelurile LLM
8. Memorie și cache
9. Securitate și măsuri de protecție
10. Integrarea MCP
- 11. Inferența SageMaker**
12. Training, Fine-Tuning și RLHF





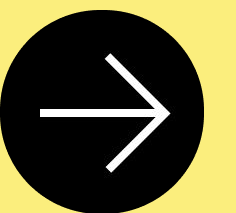
Inferența SageMaker

Lecția 11

Inferența SageMaker

- Implementarea modelelor în punctele finale SageMaker
- Implementarea conductelor de inferență în loturi
- Configurarea inferenței asincrone cu notificări
- Aspecte ale punerii în producție (monitoring, cost optimization)

Intro & Overview



Ce este Amazon SageMaker?

Platformă AWS completă pentru întregul ciclu de viață ML

SageMaker = platformă end-to-end pentru Machine Learning pe AWS

BUILD

Notebooks

Studio IDE

Data preparation

TRAIN

Training jobs

Hyperparameter tuning

Model registry

DEPLOY

Endpoints

Inference

Monitoring

Astăzi învățăm cum servim modele ML/LLM în producție pe AWS

De ce SageMaker pentru Inference?

Comparație cu alternativele AWS

 EC2 + Docker	PRO Control total asupra infrastructurii	CON Gestionezi totul manual: scaling, monitoring, updates
 Lambda	PRO Serverless, pay-per-use	CON Cold start lent, limit 15 min, limit 10 GB memory
 ECS / Fargate	PRO Containers managed, control mediu	CON Nu e optimizat pentru ML, nu are GPU autoscaling nativ
 Bedrock	PRO Zero setup, managed complet	CON Doar modele din catalog (Claude, Llama pre-built)
 SageMaker	PRO Managed + flexibil pentru ML, GPU nativ	CON Curbă de învățare AWS

SageMaker = managed infrastructure + autoscaling + GPU + multiple deployment patterns

Cele 3 tipuri de inference

Trei paradigme cu trade-off-uri diferite — alegi după use case

1

REAL-TIME

Request → Response imediat

Latency: ms — secunde

Endpoint always running

2

BATCH TRANSFORM

S3 input → procesare → S3 output

Latency: minute — ore

Compute pornit doar când rulează

3

ASYNC INFERENCE

Request → Queue → Notificare când e gata

Latency: secunde — minute

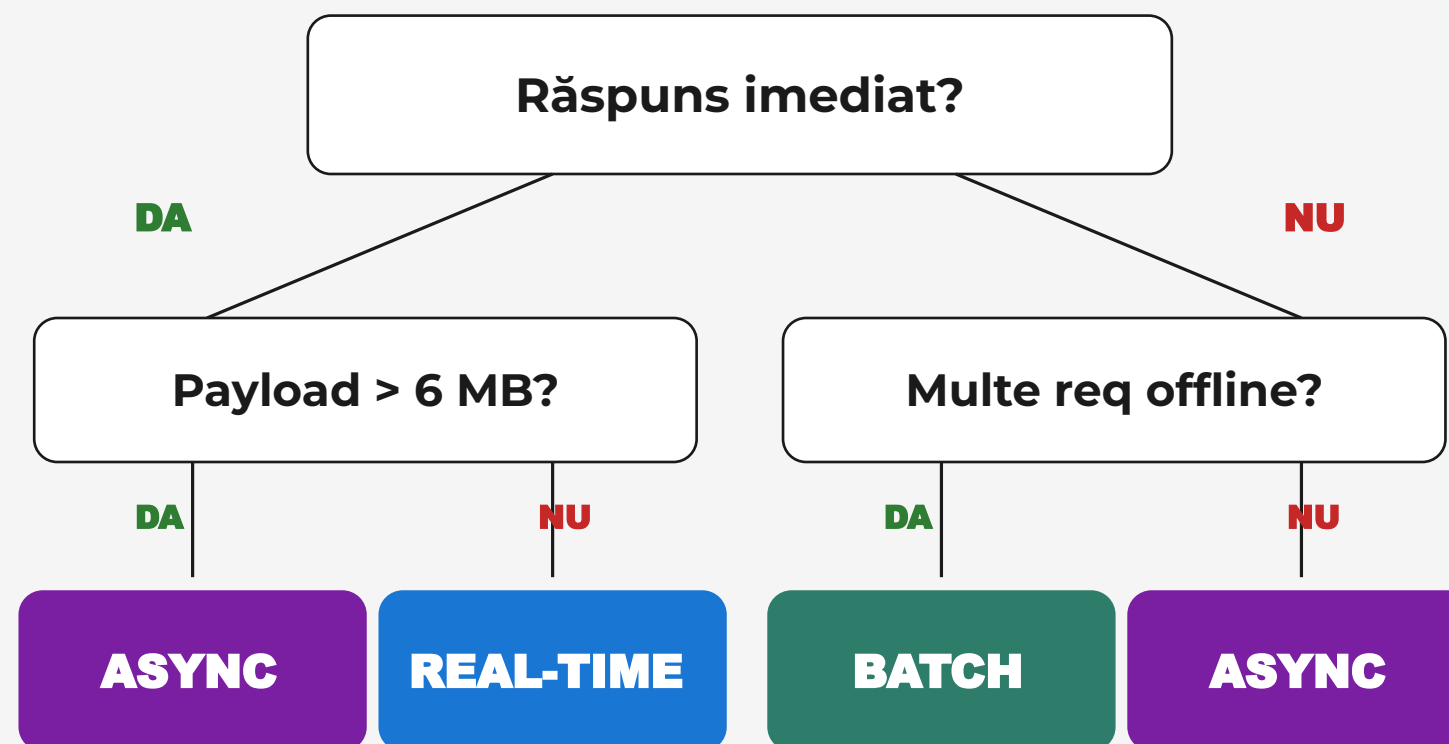
Scale-to-zero posibil

Alegerea tipului = balanță între latență, cost și pattern de trafic

Când folosești fiecare tip?

Decision flow + scenarii reale

Decision flow



Limita 6 MB e pentru sync invocations

Async acceptă până la 1 GB / request

Batch e cel mai economic pentru volume mari

Real-time = cel mai scump (instanță always-on)

Scenarii reale

API pentru aplicație web

REAL-TIME

Procesare 10.000 documente overnight

BATCH

Generare rapoarte on-demand (2-3 min)

ASYNC

Chatbot cu răspuns instant

REAL-TIME

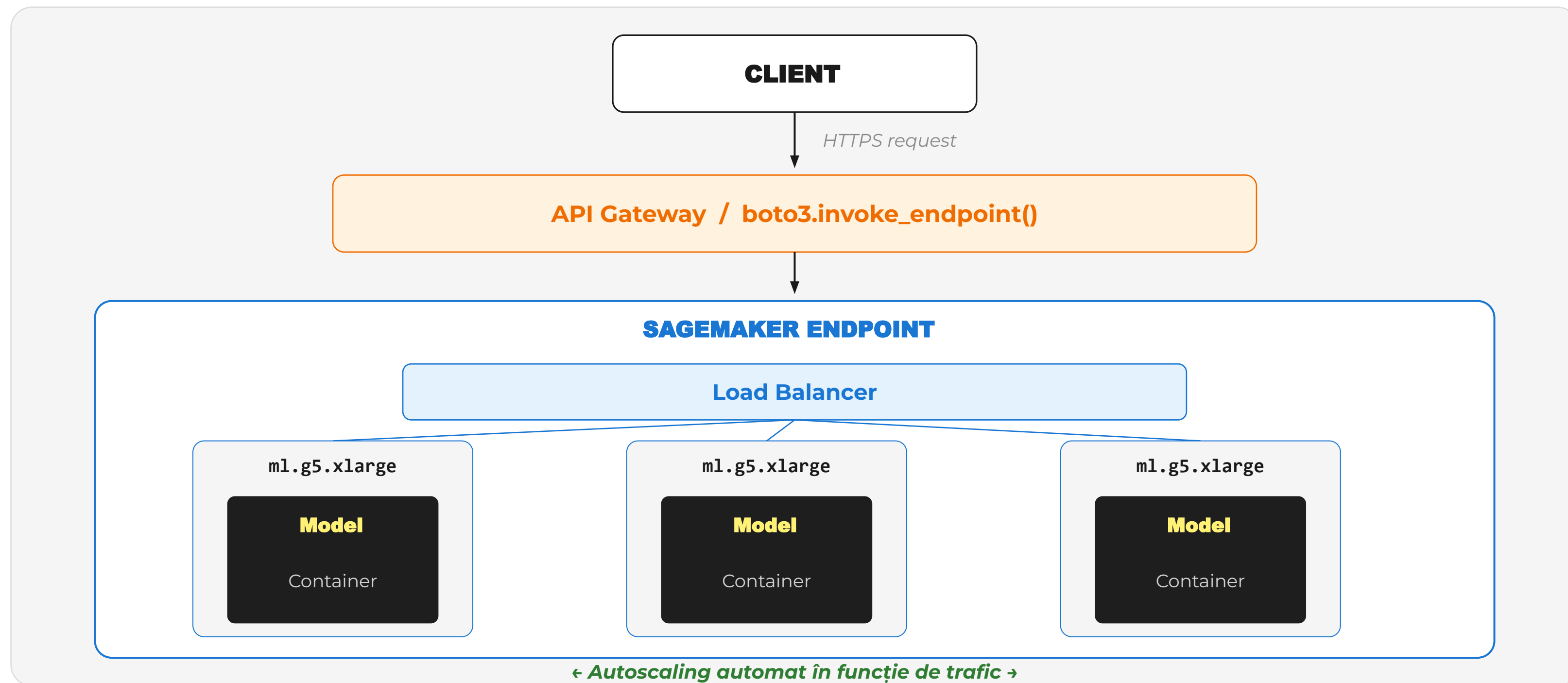
Video / audio processing (payload mare)

ASYNC

Întrebarea cheie: cât timp poate aștepta clientul răspunsul?

Arhitectura SageMaker (Real-Time)

Cum arată un endpoint din spate — managed de AWS



Tu deployezi modelul; AWS gestionează LB-ul, scaling-ul, monitoring-ul, restart-urile

Componentele SageMaker Inference

3 abstracțiuni care compun un endpoint — fiecare răspunde la o întrebare diferită

1

MODEL

CE rulează?

- Artifacts în S3 (model.tar.gz)
- Container image (ECR)
- Definește codul + greutatea modelului

2

ENDPOINT CONFIGURATION

PE CE rulează?

- Instance type (ml.g5.xlarge, ml.p4d.24xlarge)
- Instance count
- Variant weights (pentru A/B testing)

3

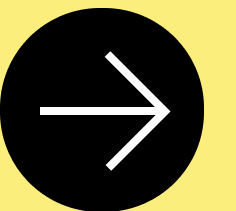
ENDPOINT

UNDE se accesează?

- URL pentru invocări (managed by AWS)
- Monitoring, logging, autoscaling
- Stable: schimbi config-ul fără downtime

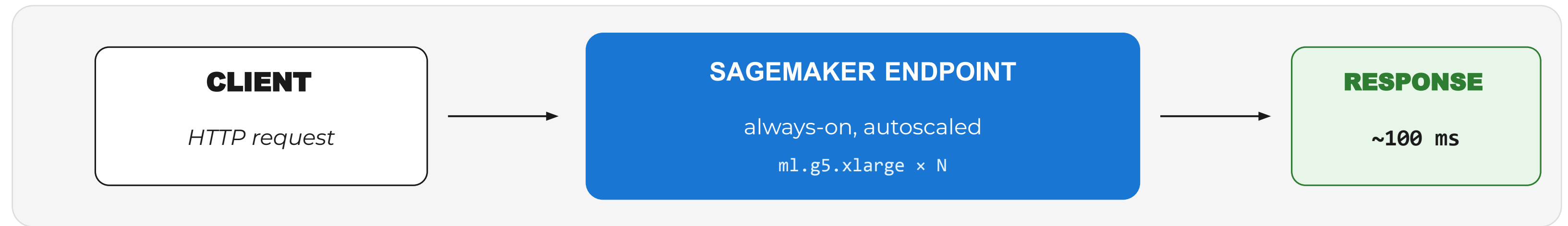
Endpoint-ul stabil; schimbi MODEL sau CONFIG → blue/green deploy fără downtime

Real-Time Inference



Real-Time Inference — Overview

Endpoint always-on, latency în milisecunde — paradigma clasică pentru API-uri și chatbots



CARACTERISTICI

- ✓ Latency scăzută (ms — secunde)
- ✓ Endpoint always-on
- ✓ Plătești pentru uptime, nu per request
- ✓ Ideal: API-uri, chatbots, aplicații interactive

LIMITE

- ✗ Payload max: **6 MB**
- ✗ Timeout: **60 secunde**
- ✗ Cost: **instanță rulează 24/7**
- ✗ Scale to 0: **imposibil**

Real-time = răspuns imediat, dar instanța rulează non-stop. Atenție la cost!

Alegerea instance type

CPU pentru modele mici și clasificare; GPU pentru LLM-uri și modele mari

CPU

modele mici, clasificare, embeddings

INSTANCE	SPECS	COST	USE CASE
ml.t3.medium	2 vCPU, 4 GB	~\$0.05/h	Dev / experiments
ml.m5.large	2 vCPU, 8 GB	~\$0.12/h	Producție generală
ml.c5.xlarge	4 vCPU, 8 GB	~\$0.20/h	CPU-intensive (embeddings)

GPU

LLM-uri, modele mari, generare

INSTANCE	SPECS	COST	USE CASE
ml.g4dn.xlarge	1× T4 (16 GB)	~\$0.74/h	Modele 7B (cu quantization)
ml.g5.xlarge	1× A10G (24 GB)	~\$1.41/h	Modele 7-13B
ml.g5.2xlarge	1× A10G (24 GB)	~\$1.52/h	Modele 13B, throughput mai mare
ml.p4d.24xlarge	8× A100 (40 GB)	~\$37/h	Modele 70B+, multi-GPU

REGULA VRAM ≈ 2× model size (FP16) — ex: Llama 7B → 14 GB → ml.g5.xlarge (24 GB)

Right-size GPU: nu ml.p4d.24xlarge pentru un model 7B! Începi mic, scalezi după măsurători.

Cod: creare endpoint real-time

HuggingFaceModel + deploy() — un singur apel creează Model + Config + Endpoint

```
1 # Container TGI optimizat pentru LLM-uri
2 tgi_image = get_huggingface_llm_image_uri("huggingface", version="2.0.1")
3
4 model = HuggingFaceModel(
5     role=ROLE_ARN,
6     image_uri=tgi_image,
7     env={
8         "HF_MODEL_ID": "TinyLlama/TinyLlama-1.1B-Chat-v1.0",
9         "SM_NUM_GPUS": "1",
10        "MAX_INPUT_LENGTH": "2048",
11        "MAX_TOTAL_TOKENS": "4096",
12    },
13 )
14
15 # deploy() creează Model + EndpointConfig + Endpoint (~5-10 min la primul deploy)
16 predictor = model.deploy(
17     instance_type="ml.g4dn.2xlarge",
18     initial_instance_count=1,
19     endpoint_name="tinylama-rt",
20 )
21
```

deploy() durează ~5-10 minute la primul deploy (download model + setup container).

Cod: invocare endpoint

2 metode — `predictor.predict()` din SDK sau `boto3 invoke_endpoint()` de oriunde

METODA 1 — PREDICTOR (SDK)

```
1 # METODA 1 – Predictor (SDK), simplu pentru notebook-uri
2 response = predictor.predict({
3     "inputs": "Explică pe scurt ce este machine learning:",
4     "parameters": {"max_new_tokens": 256, "temperature": 0.7},
5 })
6 print(response) # dict deja parsed
```

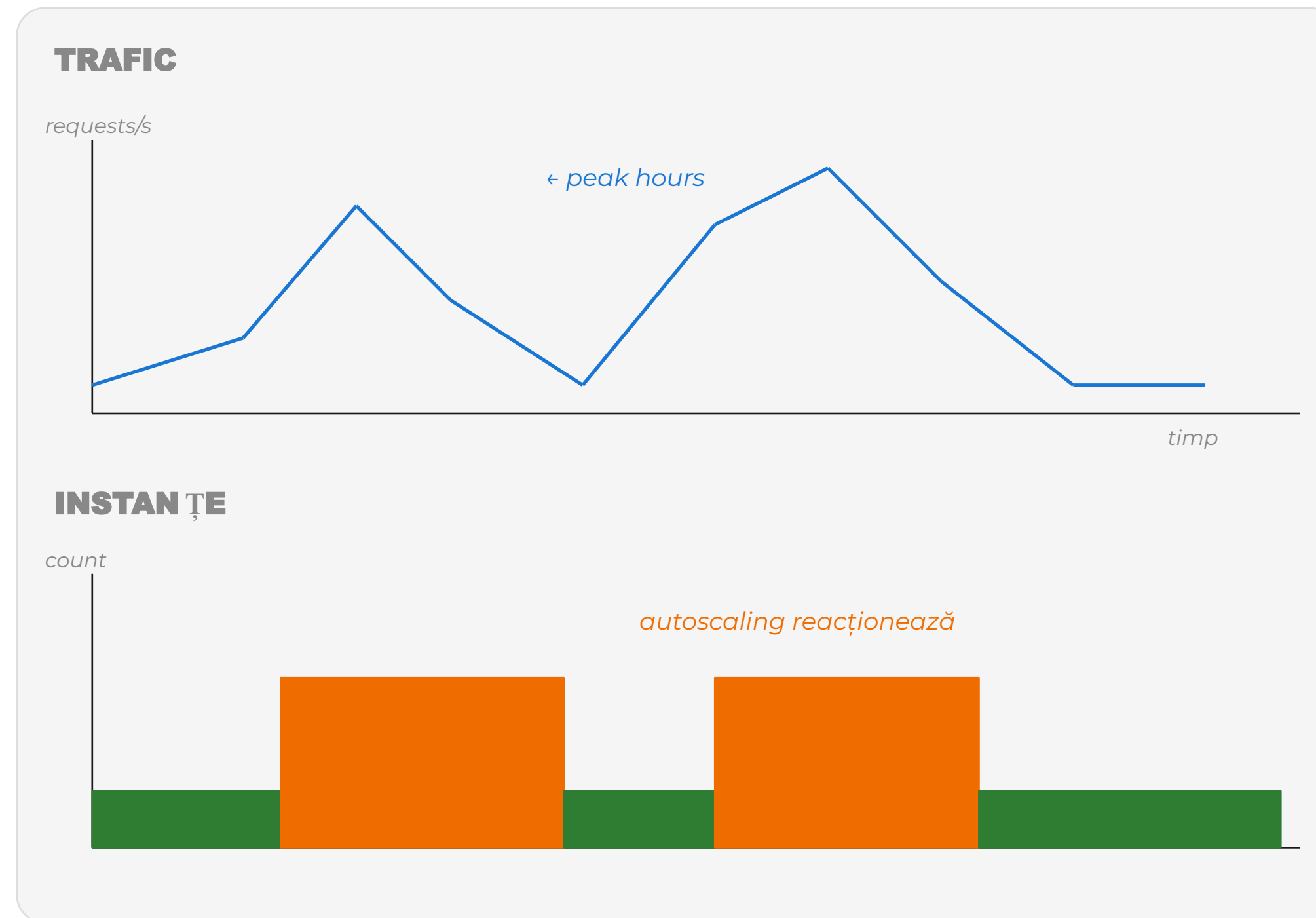
METODA 2 — BOTO3 (de oriunde)

```
1 runtime = boto3.client("sagemaker-runtime", region_name="eu-central-1")
2 response = runtime.invoke_endpoint(
3     EndpointName="tinylama-rt",
4     ContentType="application/json",
5     Body=json.dumps({"inputs": "Explică ML:"}),
6 )
7 result = json.loads(response["Body"].read()) # parsing manual
8 print(result)
9
```

Predictor = simplu pentru notebook-uri. boto3 = portabil în Lambda, ECS, oriunde.

Autoscaling — cum funcționează

Endpoint-ul scalează automat numărul de instanțe în funcție de trafic — tu setezi pragurile



METRICI PENTRU SCALING

InvocationsPerInstance **RECOMM**
requests / instanță / minut

CPUUtilization
pentru CPU instances

GPUUtilization
pentru GPU instances

ApproximateBacklogSize
doar pentru async

Default-ul AWS pentru SageMaker e `InvocationsPerInstance` — start de aici, ajustezi după observă.

Cod: configurare autoscaling

register_scalable_target stabilește limitele; put_scaling_policy spune când să scaleze

```
1 client = boto3.client("application-autoscaling", region_name="eu-central-1")
2 resource_id = "endpoint/tinyllama-rt/variant/AllTraffic"
3
4 # 1. Limite Min/Max instanțe
5 client.register_scalable_target(
6     ServiceNamespace="sagemaker",
7     ResourceId=resource_id,
8     ScalableDimension="sagemaker:variant:DesiredInstanceCount",
9     MinCapacity=1, MaxCapacity=3, # quota g4dn.2xlarge = 2, rămâi prudent
10 )
11
12 # 2. Politică: scalează după invocări/instanță/minut
13 client.put_scaling_policy(
14     PolicyName="invocations-scaling",
15     ServiceNamespace="sagemaker",
16     ResourceId=resource_id,
17     ScalableDimension="sagemaker:variant:DesiredInstanceCount",
18     PolicyType="TargetTrackingScaling",
19     TargetTrackingScalingPolicyConfiguration={
20         "TargetValue": 100, # 100 invocări/instanță/minut
21         "PredefinedMetricSpecification": {
22             "PredefinedMetricType": "SageMakerVariantInvocationsPerInstance",
23         },
24         "ScaleInCooldown": 300, "ScaleOutCooldown": 60,
25     },
26 )
27
```

Cooldown-uri: scale-in lent (5 min), scale-out rapid (1 min) — preferi over-provisioned vs latency.

Cleanup șterge resursele când termini

Endpoint-urile costă chiar dacă nu primesc requests; ordinea contează



```
1 # Metoda 1 – cu Predictor (din deploy)
2 predictor.delete_endpoint() # șterge endpoint + config
3 predictor.delete_model()
4
5 # Metoda 2 – cu boto3 (când nu mai ai obiectul predictor)
6 import boto3
7
8 sm = boto3.client("sagemaker", region_name="eu-central-1")
9
10 # Ordinea contează: endpoint → config → model
11 sm.delete_endpoint(EndpointName="tinyllama-rt")
12 sm.delete_endpoint_config(EndpointConfigName="tinyllama-rt")
13 sm.delete_model(ModelName="tinyllama-rt-model")
14
```

 **ATENȚIE**

\$1.41/h

*ml.g5.xlarge — chiar dacă
nu primește requests!*

 **PRO TIP**

Setează AWS Billing Alerts:

- Console → CloudWatch → Billing
- Alarm la \$50 / \$100 / \$200
- Email când depășești pragul

Regula de aur: orice endpoint creat în notebook trebuie șters la final. Set timer dacă uiți.

Containere optimizate: TGI vs vLLM

Containerele specializate pentru LLM oferă 3-4× throughput față de container-ul HF standard

STANDARD HuggingFace · simplu de folosit, dar **fără continuous batching** · *throughput limitat*

TGI (HuggingFace)

- ✓ Continuous batching
- ✓ Flash Attention
- ✓ Quantization (bitsandbytes, GPTQ)
- ✓ Tensor parallelism
- ✓ Streaming responses

vLLM (community)

- ✓ PagedAttention (memory eff.)
- ✓ Continuous batching
- ✓ Quantization (AWQ, GPTQ)
- ✓ Tensor parallelism
- ✓ OpenAI-compatible API

THROUGHPUT — Llama 7B pe ml.g5.xlarge (A10G)



TGI și vLLM = recomandare default pentru orice LLM în producție pe SageMaker.

Deploy cu TGI pe SageMaker

get_huggingface_llm_image_uri(backend="huggingface") → URI container TGI gata-de-folosit

```
1  # 1. URI container TGI optimizat
2  tgi_image_uri = get_huggingface_llm_image_uri("huggingface", version="2.0.1")
3
4  # 2. Model cu env vars TGI
5  model = HuggingFaceModel(
6      role=ROLE_ARN,
7      image_uri=tgi_image_uri,
8      env={
9          "HF_MODEL_ID": "TinyLlama/TinyLlama-1.1B-Chat-v1.0",
10         "SM_NUM_GPUS": "1",
11         # Optimizări TGI:
12         "MAX_INPUT_LENGTH": "2048",
13         "MAX_TOTAL_TOKENS": "4096",
14         "MAX_BATCH_TOTAL_TOKENS": "8192",
15     },
16 )
17
18 predictor = model.deploy(
19     instance_type="ml.g4dn.2xlarge", initial_instance_count=1,
20 )
21
```

QUANTIZE=bitsandbytes-nf4 → modelul 7B încapă în 6 GB VRAM (vs 14 GB FP16).

Deploy cu vLLM pe SageMaker

Același pattern ca TGI dar cu backend="vllm" — bonus: API OpenAI-compatible

CONFIG vLLM

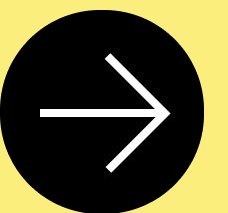
```
1 vllm_image_uri = get_huggingface_llm_image_uri("lmi")
2
3 model = HuggingFaceModel(
4     role=ROLE_ARN,
5     image_uri=vllm_image_uri,
6     env={
7         "HF_MODEL_ID": "TinyLlama/TinyLlama-1.1B-Chat-v1.0",
8         "OPTION_MAX_MODEL_LEN": "4096",
9         "OPTION_DTYPE": "bf16",
10        "OPTION_TENSOR_PARALLEL_DEGREE": "1",
11    },
12 )
13 predictor = model.deploy(
14     instance_type="ml.g4dn.2xlarge", initial_instance_count=1,
15 )
```

INVOCARE — OPENAI FORMAT

```
1 # INVOCARE — format OpenAI ChatCompletion
2 response = predictor.predict({
3     "messages": [{"role": "user", "content": "Hello!"}],
4     "max_tokens": 256,
5     "temperature": 0.7,
6 })
7 print(response) # {choices: [{message: ...}]} — la fel ca OpenAI
8
```

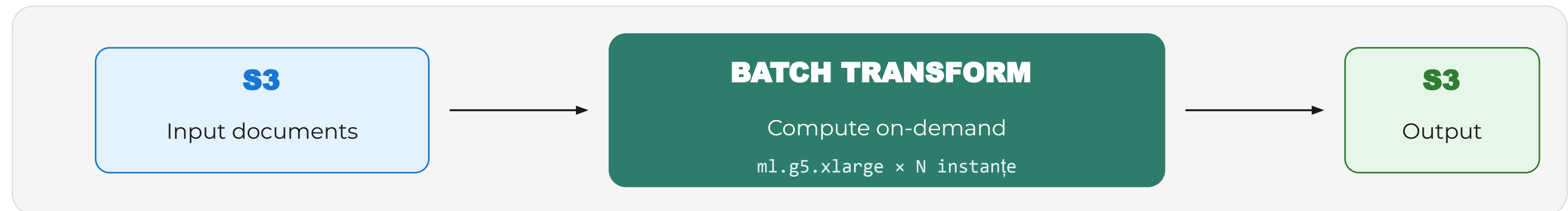
Next: Secțiunea 3 — Batch Inference (procesare offline pentru volume mari)

Batch Inference



Batch Transform

Procesare offline pentru volume mari de documente, fără endpoint persistent



CARACTERISTICI

- Procesare offline, fără endpoint persistent
- Plătești doar timpul de procesare
- Input/Output: fișiere în S3
- Async — pornești job-ul și pleci

IDEAL PENTRU

- ✓ Procesare periodică (nightly jobs)
- ✓ Re-indexare documente RAG
- ✓ Embeddings pentru mii de docs
- ✓ Clasificare / etichetare în bulk

COST Real-time always-on ~\$1000/lună (m1.g5.xl) vs Batch on-demand ~\$5 pentru 100k docs

Batch Transform = pay-per-job, nu pay-per-time. Endpoint-ul se distruge după procesare.

Structura Input / Output în S3

Convenția SageMaker: fiecare fișier = un request, output-ul reflectă structura input-ului

INPUT BUCKET

s3://my-bucket/batch-input/

- |— request_001.json
- |— request_002.json
- |— request_003.json
- |— request_004.json
- |— ...

Fiecare fișier = un request independent

```
{
  "inputs": "Extrage datele din factură...",
  "parameters": {"max_new_tokens": 512}
}
```



transform

OUTPUT BUCKET

s3://my-bucket/batch-output/

- |— request_001.json.out
- |— request_002.json.out
- |— request_003.json.out
- |— request_004.json.out
- |— ...

Generat automat — 1:1 cu input-ul

```
{
  "generated_text": "Total: 1.250 RON...",
  "tokens": 147
}
```

Convenția: <input>.json → <input>.json.out — păstrezi mapping-ul 1:1

Cod: pornește un Batch Transform

3 pași — Model → Transformer → start job (async, nu blochează)

```
1  ROLE_ARN = "<EXECUTION_ROLE_ARN>" # local: ARN explicit, nu get_execution_role()
2
3  # Pasul 1 – definim modelul (FĂRĂ deploy!)
4  model = HuggingFaceModel(
5      role=ROLE_ARN,
6      transformers_version="4.37",
7      pytorch_version="2.1",
8      py_version="py310",
9      env={"HF_MODEL_ID": "TinyLlama/TinyLlama-1.1B-Chat-v1.0",
10         "HF_TASK": "text-generation"},
11  )
12
13  # Pasul 2 – configurăm transformer-ul
14  transformer = model.transformer(
15      instance_count=1, instance_type="ml.g4dn.2xlarge",
16      output_path="s3://my-bucket/batch-output/",
17      strategy="SingleRecord", # un fișier = un request
18      max_payload=6, max_concurrent_transforms=4,
19      accept="application/json",
20  )
21
22  # Pasul 3 – start job (async, nu blochează)
23  transformer.transform(
24      data="s3://my-bucket/batch-input/",
25      content_type="application/json", wait=False,
26  )
27
```

wait=False → tu pleci, AWS rulează job-ul. Verifici status-ul cu describe_transform_job().

Monitorizare Job-ului

Verifici status-ul, descarci rezultatele, urmărești metrici în CloudWatch

```
1 sm = boto3.client("sagemaker", region_name="eu-central-1")
2
3 # Verificăm status-ul job-ului
4 job_name = transformer.latest_transform_job.name
5 response = sm.describe_transform_job(TransformJobName=job_name)
6 status = response["TransformJobStatus"]
7 # InProgress | Completed | Failed | Stopping | Stopped
8
9 # Sau așteaptă blocant până se termină
10 transformer.wait()
11
12 # După finalizare, descărcăm un rezultat
13 s3 = boto3.client("s3", region_name="eu-central-1")
14 s3.download_file(
15     "my-bucket",
16     "batch-output/request_001.json.out",
17     "local_result.json",
18 )
19
```

CLOUDWATCH METRICS

Invocations

număr requests procesate

InvocationErrors

requests eșuate

DataProcessed

volum total în bytes

TransformLatency

time / request

wait() = blocant; describe_transform_job + polling = neblocant; logs și metrici live în CloudWatch.

Optimizări Batch Transform

3 pârgii pentru viteza și costul job-ului

1

PARALELISM

Mai multe instanțe + mai multe requests simultan per instanță.

`instance_count=4` → 4 mașini procesează în paralel

`max_concurrent_transforms=4` → 4 requests/instanță simultan

CÂȘTIG

Throughput × 16

2

MULTE INSTANȚE MICI

Pentru batch mare, mai multe instanțe mici bat o instanță uriașă.

1× `ml.g5.4xlarge` = \$2.27/h, secvențial pe 1 mașină

4× `ml.g5.xlarge` = \$5.64/h, dar termini de 4× mai repede

CÂȘTIG

Latency total ↓

3

SPLIT INTELIGENT

Cum sunt împărțite request-urile în interiorul fișierelor.

`strategy="SingleRecord"` → fiecare fișier = 1 request

`strategy="MultiRecord" + split_type="Line"` → multe linii / fișier

CÂȘTIG

Overhead ↓

Combini cele 3 pârgii — test cu un sample mic înainte să rulezi pe tot dataset-ul.

Spot Instances — economie 70%

Folosești capacitatea AWS neutilizată — ideal pentru batch (work se reia automat)

```
1 model = HuggingFaceModel(  
2     role=ROLE_ARN,  
3     transformers_version="4.37", pytorch_version="2.1", py_version="py310",  
4     env={"HF_MODEL_ID": "TinyLlama/TinyLlama-1.1B-Chat-v1.0",  
5         "HF_TASK": "text-generation"},  
6 )  
7  
8 transformer = model.transformer(  
9     instance_count=2,  
10    instance_type="ml.g4dn.2xlarge",  
11    output_path="s3://my-bucket/output/",  
12  
13    # Spot configuration — economie ~70% vs on-demand  
14    use_spot_instances=True,  
15    max_retry_attempts=2, # retry dacă e întrerupt  
16 )  
17  
18 transformer.transform(  
19     data="s3://my-bucket/input/",  
20     content_type="application/json",  
21     wait=True,  
22 )  
23
```

ECONOMIE

~70%

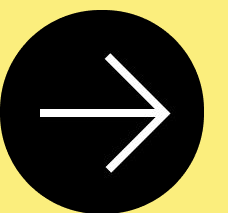
vs. on-demand

TRADE-OFF

- Job-ul poate fi întrerupt
- AWS îl repornește automat
- max_retry_attempts limitează re-run-urile
- OK pentru batch — NU pentru real-time

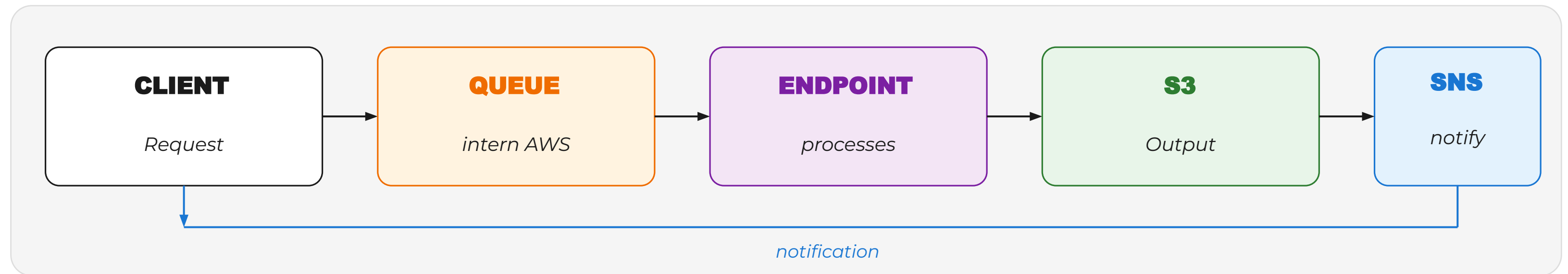
Next: Secțiunea 4 — Async Inference (queue-based + scale-to-zero)

Async Inference



Async Inference — Overview

Queue-based: trimiți request-ul, primești output location, primești notificare când e gata



CARACTERISTICI

Payload max: **1 GB** vs 6 MB la real-time

Timeout: **15 min** vs 60s la real-time

Response: **OutputLocation** → poll sau SNS

Scale-to-zero: **DA** MinCapacity=0

IDEAL PENTRU

- ✓ Payloads mari (imagini, audio, PDF-uri)
- ✓ Procesări lungi (2-15 minute)
- ✓ Trafic imprevizibil — perioade fără requests
- ✓ Workloads sporadice cu cost-sensitive

Async = real-time + queue + payloads mari + scale-to-zero. Plătești doar când rulează.

Comparație: Real-Time vs Async vs Batch

5 dimensiuni de comparație — alegi pe baza pattern-ului de trafic

	REAL-TIME	ASYNC	BATCH
Latency	ms — 60s	secunde — 15 min	minute — ore
Payload	6 MB	1 GB	Nelimitat
Cost model	Per uptime	Per uptime*	Per job
Scale to 0	NU	DA	N/A
Response	Sync	Poll / SNS	S3

* Async poate scala la 0 → cost = 0 când nu e trafic

DOCUMENT ANALYST — exemple concrete

Chat cu agentul	REAL-TIME	Upload PDF mare → extracție	ASYNC	Reindexare nightly 10k docs	Coco
-----------------	-----------	-----------------------------	-------	-----------------------------	------

Întrebarea cheie: cât de mare e payload-ul și cât poate aștepta clientul?

Cod: deploy async endpoint

AsyncInferenceConfig + SNS notifications + deploy

```
1 # 1. Configurăm comportamentul async
2 async_config = AsyncInferenceConfig(
3     output_path="s3://my-bucket/async-output/",
4
5     # Notificări SNS (opțional)
6     notification_config={
7         "SuccessTopic": "arn:aws:sns:eu-central-1:123:success",
8         "ErrorTopic": "arn:aws:sns:eu-central-1:123:error",
9     },
10    max_concurrent_invocations_per_instance=4,
11 )
12
13 # 2. Deploy cu config-ul async (model = HuggingFaceModel cu TGI, ca la real-time)
14 predictor = model.deploy(
15     instance_type="ml.g4dn.2xlarge",
16     initial_instance_count=1,
17     async_inference_config=async_config,
18     endpoint_name="tinyllama-async",
19 )
```

Endpoint-ul rulează exact ca real-time, dar acceptă requests prin queue în loc de sync.

Invocare și polling pentru rezultat

invoke_endpoint_async returnează imediat → polling S3 pentru output

```
1 runtime = boto3.client("sagemaker-runtime", region_name="eu-central-1")
2 s3 = boto3.client("s3", region_name="eu-central-1")
3
4 # 1. Trimite – returnează imediat
5 response = runtime.invoke_endpoint_async(
6     EndpointName="tinyllama-async",
7     ContentType="application/json",
8     InputLocation="s3://my-bucket/async-input/req.json",
9 )
10 output_loc = response["OutputLocation"]
11
12 # 2. Polling S3 pentru rezultat
13 bucket, key = output_loc.replace("s3://", "").split("/", 1)
14 while True:
15     try:
16         result = s3.get_object(Bucket=bucket, Key=key)
17         break
18     except s3.exceptions.NoSuchKey:
19         time.sleep(5)
```

FLOW

1

Invoke async

Trimiți request, răspuns instant cu OutputLocation

2

Wait

Endpoint-ul procesează în queue

3

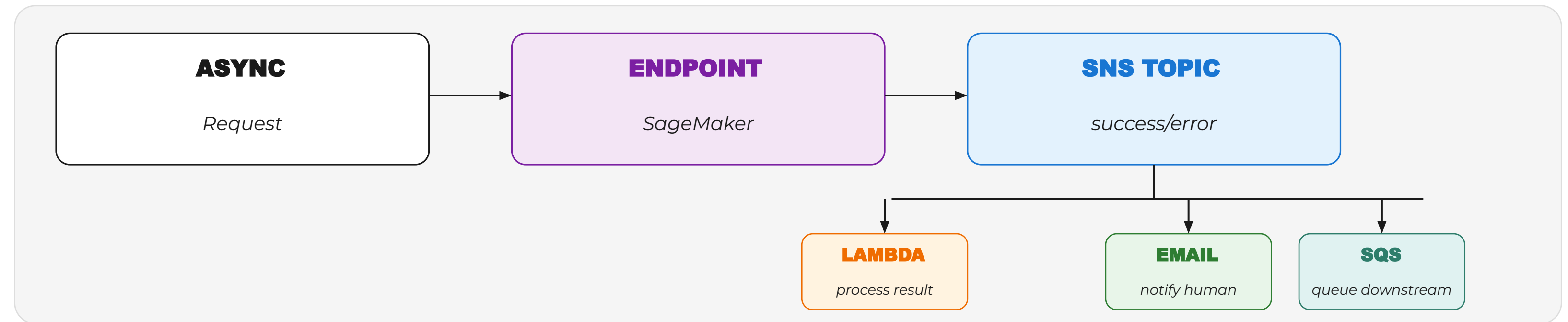
Fetch S3

Polling sau SNS notification → citești output

Polling = simplu dar costă API calls; SNS = scalabil dar necesită Lambda/SQS.

SNS Notifications Setup

Endpoint trimite mesaj în SNS Topic → fan-out la Lambda, Email, SQS



```
1 def lambda_handler(event, context):
2     for record in event["Records"]:
3         msg = json.loads(record["Sns"]["Message"])
4         if msg["invocationStatus"] == "Completed":
5             process_result(msg["responseParameters"]["outputLocation"])
6         else:
7             handle_error(msg["failureReason"])
8
```

SNS = decuplaj. Endpoint-ul nu știe cine consumă; consumatorii nu știu cine produce.

Scale-to-Zero — trade-offs

Cost zero la idle vs cold start lent — alegi în funcție de SLA

✓ AVANTAJE

Cost \$0 la idle

Plătești doar când rulează

Workloads sporadice

Trafic imprevizibil = OK

Scale automat la peak

Pornește instanțe când vine load

✗ DEZAVANTAJE

Cold start: 5-10 min

Primul request după idle e foarte lent

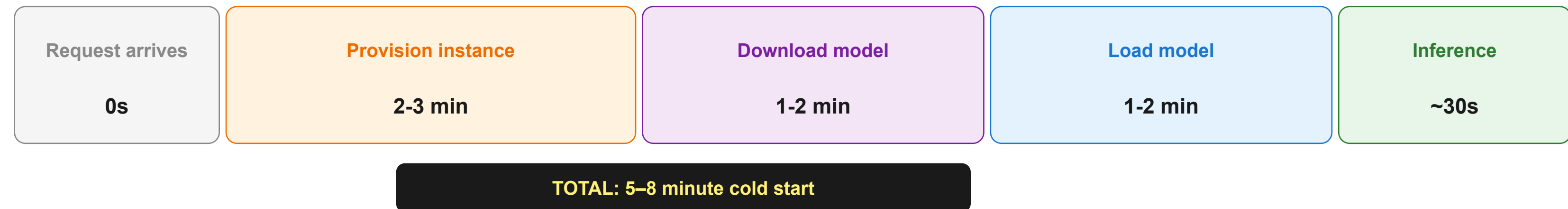
Model trebuie reîncărcat

De pe S3 — durează 1-2 min

Nu pentru SLA strict

Latency-sensitive workloads → MinCapacity=1

COLD START TIMELINE



Next: Secțiunea 5 — Document Analyst Deployment Options